AI Engineering Environment Setup

Foundation Session 1

Mr. Shridhar Galande

Session Roadmap

What we'll cover in this session

01 AI Engineering Environment Fundamentals

02 Python & Virtual Environment Setup

03 IDE, Jupyter & VS Code Setup

04 GPU, CUDA & Runtime Awareness

05 Cloud Notebooks & GPU Platforms

06 Installing GenAI Libraries

07 APIs, Keys & Endpoint Setup

08 Running First LLM Application

09 Troubleshooting & Best Practices

SECTION 1 OF 9

AI Engineering Environment Fundamentals

What is AI Engineering Setup?

- **The practice of configuring the tools, dependencies, and compute needed to build AI/GenAI applications**
- **Goes beyond writing code:**
  - Choosing the right hardware (CPU vs GPU)
  - Managing library versions and dependencies
  - Deciding where code runs — locally or in the cloud
  - Securing credentials for hosted model access
- **A well-set-up environment is the foundation for reliable experimentation and deployment**

Traditional vs GenAI Environments

Traditional Software

- ✓ Lightweight dependencies
- ✓ Runs comfortably on any laptop
- ✓ Few version conflicts
- ✓ Minimal hardware requirements
- ✓ Predictable, stable installs
- ✓ Deployment rarely needs a GPU

GenAI / LLM Projects

- ✓ Large, fast-changing library ecosystem
- ✓ Often needs GPU acceleration
- ✓ Frequent version incompatibilities
- ✓ High memory & VRAM requirements
- ✓ Cloud or hybrid execution common
- ✓ Model weights add multi-GB storage needs

CPU vs GPU · Local vs Cloud

CPU vs GPU

- ✓ CPU: general-purpose, sequential tasks
- ✓ GPU: thousands of cores, parallel math
- ✓ Training & inference on large models needs GPU speed
- ✓ Small demos can still run on CPU
- ✓ GPUs are measured in VRAM, not just clock speed

Local vs Cloud Execution

- ✓ Local: full control, limited hardware
- ✓ Cloud: scalable GPUs, pay-as-you-go
- ✓ Google Colab: free GPU access for learning
- ✓ Choice depends on project size & budget
- ✓ Hybrid: prototype locally, scale training in the cloud

Why Dependency Management Matters

- **GenAI libraries evolve quickly — versions of PyTorch, Transformers, and CUDA must stay compatible**
- **Typical failures in GenAI projects:**
  - Installing a library that silently breaks another
  - CUDA version mismatch with GPU drivers
  - Notebook kernels crashing from memory overload
  - “Works on my machine” issues between team members
- **Isolated environments (covered next) are the primary defense against these failures**

Verifying Your Environment

- **Before writing any AI code, confirm the basics are in place**
- **A quick verification checklist:**
  - Confirm the Python interpreter and version in use
  - Confirm pip is available and up to date
  - Confirm whether a GPU is visible to your system
- **Running these checks first saves hours of confusing downstream errors**

TERMINAL / NOTEBOOK

```
python --version
pip --version
nvidia-smi        # shows GPU info, if available
```

SECTION 2 OF 9

Python & Virtual Environment Setup

Python Installation & Pip Basics

- **Python is the primary language for AI/ML — version 3.9+ recommended for GenAI work**
- **pip is Python's package manager:**
  - pip install <package> — installs a library
  - pip list — shows installed packages
  - pip freeze > requirements.txt — captures exact versions
- **Keeping pip itself updated avoids many installation errors**

CHECK & UPDATE PIP

```
python -m pip install --upgrade pip
pip install requests
```

Checking & Changing Your Python Version

- **Multiple Python versions can exist side by side on one machine**
- **To check your active version:**
  - python --version or python3 --version
  - which python (macOS/Linux) or where python (Windows) shows the active path
- **To switch versions:**
  - Create a venv using a specific interpreter binary
  - Or use conda / uv to install and pin a Python version per project

SWITCH PYTHON VERSION

```
python3.11 -m venv env       # venv with a specific version
conda create -n ai python=3.11
uv python install 3.11
```

Virtual Environments

- **A virtual environment is an isolated Python installation just for one project**
- **Why it matters:**
  - Prevents one project's packages from breaking another's
  - Lets different projects use different library versions
  - Keeps your system-wide Python clean
- **Always activate the environment before installing anything**

CREATE & ACTIVATE A VENV

```
python -m venv env
source env/bin/activate          # macOS/Linux
env\Scripts\activate             # Windows
```

Package Isolation & Dependency Conflicts

- **Dependency conflicts happen when two packages need incompatible versions of the same library**
- **How isolation helps:**
  - Each virtual environment has its own package set
  - Conflicts in one project don't affect others
  - Easier to reproduce a working setup on another machine
- **Best practice: one virtual environment per project, documented in requirements.txt**

REPRODUCE AN ENVIRONMENT

```
pip freeze > requirements.txt
pip install -r requirements.txt
```

Docker — A Brief Introduction

- **Docker packages code, dependencies, and system settings into a single portable “container”**
- **Why it's useful for AI engineering:**
  - Guarantees the same environment across machines
  - Avoids “works on my machine” problems entirely
  - Widely used to deploy GenAI applications to production
- **For this course, it's enough to recognize Docker's role — deep usage comes later in the program**

BASIC DOCKER COMMANDS

```
docker build -t ai-app .
docker run -p 8000:8000 ai-app
```

Git & Version Control for AI Projects

- **Git tracks changes to your code over time and enables safe collaboration**
- **Why it matters for AI/GenAI work:**
  - Experiment freely — Git lets you roll back to a working version anytime
  - Branches let you try new approaches without breaking what already works
  - Essential once notebooks and scripts are shared across a team
- **What NOT to commit in AI projects:**
  - Virtual environments (env/, venv/)
  - Model weights, checkpoints, and the Hugging Face cache
  - API keys, tokens, and .env files

GIT BASICS + .gitignore

```
git init
git add .
git commit -m "Initial commit"
# .gitignore should include: env/  .env  *.ipynb_checkpoints/
```

HANDS-ON LAB I

Virtual Environment Walkthrough

Create a virtual environment, activate it, and install a sample package to see isolation in action.

Open notebook → **Section 3 — Virtual Environment Walkthrough**

SECTION 3 OF 9

IDE, Jupyter & VS Code Setup

Why an IDE Matters & Jupyter Workflow

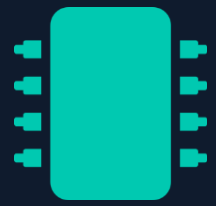
- **A good IDE speeds up debugging, navigation, and collaboration**
- **Jupyter Notebook is the standard tool for AI/ML experimentation:**
  - Mixes code, output, and narrative explanation in one document
  - Runs code cell-by-cell for fast, iterative testing
  - Ideal for exploring data and visualizing model outputs
- **We'll explore Jupyter hands-on at the start of today's notebook**

RUN JUPYTER

```
jupyter notebook
# or, for the newer interface:
jupyter lab
```

VS Code Setup & Python Extensions

- **Visual Studio Code is a lightweight, extensible editor widely used for AI development**
- **Key setup steps:**
  - Install the Python extension (ms-python.python) for linting & IntelliSense
  - Install the Jupyter extension (ms-toolsai.jupyter) to run notebooks inside VS Code
  - Select the correct Python interpreter (Ctrl/Cmd+Shift+P → “Python: Select Interpreter”)
- **VS Code + Jupyter gives you the best of both worlds: notebook flexibility with IDE tooling**



SECTION 4 OF 9

GPU, CUDA & Runtime Awareness

GPU Acceleration · VRAM vs RAM

What is GPU Acceleration?

- ✓ GPUs run thousands of operations in parallel
- ✓ Ideal for matrix math used in neural networks
- ✓ Drastically reduces training & inference time
- ✓ Why LLMs specifically need GPUs to run efficiently
- ✓ Measured in CUDA cores and compute capability

VRAM vs RAM

- ✓ RAM: system memory for general programs
- ✓ VRAM: dedicated GPU memory for model weights
- ✓ Running out of VRAM causes OOM (out-of-memory) errors
- ✓ Larger models need more VRAM — a key sizing factor
- ✓ Quantization reduces VRAM needs at some accuracy cost

CUDA Basics & GPU Usage in Colab

- **CUDA is NVIDIA's platform that lets software talk directly to the GPU**
- **Key ideas for beginners:**
  - PyTorch/TensorFlow use CUDA under the hood for GPU acceleration
  - CUDA version must match your GPU driver and library versions
  - In Google Colab: Runtime → Change runtime type → select GPU
- **You don't need to install CUDA yourself in Colab — it's pre-configured**

CHECK GPU FROM PYTHON

```
import torch
torch.cuda.is_available()       # True/False
torch.cuda.get_device_name(0)   # e.g., 'Tesla T4'
```

SECTION 5 OF 9

Cloud Notebooks & GPU Platforms

Why Cloud Platforms & Free GPU Options

- **Cloud notebooks remove the need for expensive local hardware**
- **Popular free/low-cost GPU options:**
  - Google Colab — free GPU/TPU access, browser-based
  - Kaggle Notebooks — free GPU quota for competitions & learning
  - Paid cloud GPUs (AWS, GCP, Azure) for production-scale work
- **This program primarily uses Google Colab for hands-on labs**

Colab Basics, Runtime Selection & Google Drive

- **Google Colab basics:**
  - Runtime → Change runtime type → choose GPU (or TPU)
  - Cells run just like a local Jupyter notebook
  - Sessions can disconnect after inactivity — save often
- **Connecting Google Drive lets you persist files, datasets, and models across sessions**

MOUNT GOOGLE DRIVE

```
from google.colab import drive
drive.mount('/content/drive')
```

HANDS-ON LAB II

Colab GPU Runtime & Google Drive

Open Google Colab, select a GPU runtime, and mount Google Drive to persist files.

Open notebook → **Section 4 — Colab GPU Runtime & Google Drive**

SECTION 6 OF 9

Installing GenAI Libraries

Transformer Ecosystem & PyTorch Basics

- **The Hugging Face transformers library gives easy access to thousands of pretrained models**
- **PyTorch basics:**
  - The most widely used deep learning framework in research
  - Provides tensors, autograd, and GPU support
  - transformers is built on top of PyTorch (and TensorFlow)
- **Together, these two libraries power most of today's GenAI applications**

Installing Libraries & Version Compatibility

- **Version compatibility matters:**
  - PyTorch version must match your CUDA version (for GPU use)
  - transformers releases can introduce breaking changes
  - Always check documentation when errors mention version mismatches
- **Pinning versions in requirements.txt avoids surprises later**

INSTALL COMMANDS

```
pip install torch transformers
# GPU build matched to a CUDA version:
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

SECTION 7 OF 9

APIs, Keys & Endpoint Setup

What Are API Keys? Endpoint Basics

- **An API key is a unique credential that authenticates your requests to a hosted service**
- **Endpoint basics:**
  - An endpoint is a URL where a hosted model receives requests
  - Hosted LLMs (OpenAI, Groq, Anthropic, etc.) expose endpoints for chat, completion, and embeddings
  - Requests typically include your API key and the input prompt

Hosted LLMs, Authentication & Environment Variables

- **Authentication confirms who is making a request, using your API key**
- **Environment variables keep secrets out of your code:**
  - Store keys using os.environ or a .env file, never hard-coded
  - getpass lets you enter a key securely inside a notebook
  - Never commit API keys to shared or public repositories

SAFE KEY HANDLING

```
import os
api_key = os.environ.get("MY_API_KEY")
from getpass import getpass
api_key = getpass("Enter your API key: ")
```

SECTION 8 OF 9

Running First LLM Application

Loading Transformer Models & Running Inference

- **Loading a pretrained model takes just a few lines using transformers pipelines**
- **Inference workflow:**
  - Load a tokenizer and model (e.g., distilgpt2)
  - Convert text input into tokens the model understands
  - Run a forward pass to generate an output
- **This is the same basic pattern used across almost all GenAI applications**

LOAD A MODEL

```
from transformers import pipeline
generator = pipeline("text-generation", model="distilgpt2")
```

Prompt-Based Generation Workflow

- **A prompt is the text input you give a model to guide its output**
- **Basic text generation workflow:**
  - Write a clear prompt describing the desired output
  - Pass it to the loaded pipeline, which tokenizes, generates, and decodes the output for you
  - Review and iterate on the output — prompting is experimental
- **We'll run this live in the notebook using a small pretrained model**

GENERATE TEXT

```
output = generator("The future of AI engineering is", max_length=40)
print(output[0]["generated_text"])
```

Hugging Face Tokens & Authentication

- **Many models on the Hugging Face Hub are gated or require authentication to download**
- **Getting a token:**
  - Create a free account at huggingface.co
  - Generate an access token under Settings → Access Tokens
- **Using your token:**
  - Log in once via the CLI, or authenticate directly inside a notebook
  - Never hard-code a token directly in your notebook or commit it to Git

AUTHENTICATE WITH HUGGING FACE

```
huggingface-cli login
# or, inside a notebook:
from huggingface_hub import login
login(token="hf_...")
```

Model Caching with Hugging Face

- **Downloaded models are cached locally so you don't re-download them on every run**
- **Default cache location:**
  - ~/.cache/huggingface/hub on macOS/Linux
  - C:\Users\<name>\.cache\huggingface\hub on Windows
- **Why this matters:**
  - First run downloads the model — can take a while for larger models
  - Later runs load instantly straight from cache
  - Clear the cache if you're running low on disk space

MANAGE THE CACHE

```
export HF_HOME=/path/to/custom/cache   # change cache location
rm -rf ~/.cache/huggingface/hub        # clear cache manually
```

SECTION 9 OF 9

Troubleshooting & Best Practices

Common Errors in GenAI Projects

- CUDA mismatch — GPU driver, CUDA toolkit, and library versions don't align
- Package conflicts — two libraries require incompatible dependency versions
- OOM (out-of-memory) errors — model or batch size exceeds available VRAM/RAM
- Broken kernels — notebook kernel crashes or becomes unresponsive

TYPICAL ERROR MESSAGE

```
RuntimeError: CUDA out of memory. Tried to allocate 2.10 GiB
(GPU 0; 14.75 GiB total capacity; 12.1 GiB already allocated)
```

Best Practices & Notebook Restart Discipline

- **Dependency troubleshooting:**
  - Read the full error message — it usually names the conflicting package
  - Search the exact error text before trying random fixes
  - Reinstall in a fresh virtual environment if conflicts pile up
- **Notebook restart practices:**
  - Restart the kernel after major code or environment changes
  - Run cells top-to-bottom after a restart to confirm a clean state

QUICK FIXES

```
pip show <package>          # check installed version
pip install <package> --upgrade
```

Key Takeaways

- GenAI environments differ from traditional software setups in dependency complexity, GPU needs, and runtime awareness
- Virtual environments and dependency isolation prevent conflicts across AI/ML projects
- Jupyter and VS Code together form the core AI engineering workflow
- GPUs, CUDA, and cloud platforms like Colab remove the hardware barrier for beginners
- API keys, endpoints, and environment variables connect your code to hosted LLMs securely
- You've run inference on a real transformer model end-to-end
- Most environment issues follow recognizable patterns — troubleshooting is a learnable skill
- Reproducibility (requirements.txt, version pinning) is what makes a setup shareable and durable

Thank You

Next up: Mathematical Foundations for AI Systems